

FGV EMap

João Pedro Jerônimo e Eduardo Adame

Aprendizado de Máquina

Revisão para A1

Rio de Janeiro

2026

Nota: Esse resumo é uma adaptação das notas da disciplina disponibilizadas pelo veterano Eduardo Adame junto de adições feitas por João Pedro Jerônimo, para acessar as notas originais, acesse aqui. Eu também estou fazendo um repositório contendo modelos de machine learning que estão sendo estudados nessa disciplina, para acessar o repositório, clique aqui

Conteúdo

1	Método dos Vizinhos mais próximos (k-NN)	3
1.1	Introdução Intuitiva	4
1.2	Classificação	4
1.3	Regressão	5
1.4	Qual distância escolher?	5
1.4.1	Normalização para média zero e variância um	6
1.4.2	Similaridade Cosseno	6
1.4.3	Aprendendo Métricas	6
1.5	Maldição da Dimensionalidade	7
1.5.1	Manifolds de baixa dimensão	7
2	Regressão Linear	8
2.1	O Problema	9
2.2	Perspectiva Probabilística	11
2.3	Modelo com expansão de base	12
2.3.1	Polinômios	13
2.3.2	Funções de base radiais	14
3	Regressão Logística	17
3.1	Regressão Logística Bayesiana	20
3.1.1	Aproximação de Laplace	22
3.2	Problemas multiclasse, classificador <i>softmax</i>	23
4	Redes Neurais	24
4.1	Estrutura Inicial	25
4.2	Estrutura Matemática	26
4.3	Não-linearidade	26
4.4	Treinamento	26
4.4.1	Forward Pass	27
4.4.2	Backpropagation	27
4.4.2.1	Regressão	27
4.4.2.2	Camadas Escondidas	28
4.4.3	O Algoritmo Completo	32
4.4.4	Exemplo em código	32
5	Inferência Variacional	33
5.1	Introdução	34
5.2	Propriedades da divergência de Kullback-Leibler	34
5.3	Cota Inferior (ELBO)	36
5.4	Lidando com ELBO intratável	37

Método dos Vizinhos mais próximos (k-NN)

1.1 Introdução Intuitiva

O método dos vizinhos mais próximos (k-NN) é um algoritmo de aprendizado de máquina simples e eficaz usado para classificação e regressão. Ele funciona com base na ideia de que objetos semelhantes estão próximos uns dos outros no espaço de características. Para classificar um novo ponto, o k-NN identifica os k pontos mais próximos no conjunto de treinamento e atribui a classe mais comum entre esses vizinhos ao novo ponto.

O valor de k é um hiperparâmetro que pode ser ajustado para melhorar o desempenho do modelo. O k-NN é fácil de entender e implementar, mas pode ser computacionalmente caro para grandes conjuntos de dados, pois requer o cálculo das distâncias entre o novo ponto e todos os pontos do conjunto de treinamento.

Um das hipóteses fundamentais em ML é que existe algum nível de suavidade no mapeamento entre o espaço de entrada $\mathcal{X}$ e o de saída $\mathcal{Y}$. Em outras palavras, se dois elementos $x, x' \in \mathcal{X}$ são semelhantes, então eles devem ter saídas $y, y' \in \mathcal{Y}$ similares. O método dos k vizinhos mais próximos (k nearest neighbors, k-NN), proposto por Cover e Hart (1967), aplica diretamente esse conceito.

Nesse capítulo, vamos estudar como k-NN pode ser usados para problemas de classificação e regressão. Discutiremos o impacto do k e também da escolha de distância (ou métrica) para o espaço $\mathcal{X}$, que é primordial para a aplicação do método. Finalmente, estudaremos o comportamento do método k-NN quando a dimensionalidade do espaço $\mathcal{X}$ é alta

1.2 Classificação

Seja $\mathcal{D} := \{(x_1, y_1), \dots, (x_N, y_N)\} \subset \mathcal{X} \times \mathcal{Y}$ o conjunto de treinamento e $d : \mathcal{D} \times \mathcal{D} \rightarrow \mathbb{R}^+ \cup \{0\}$ uma função de **distância**. Vamos supor que queremos classificar um vetor $x \in \mathcal{X}$ arbitrário.

O método k-NN classifica o vetor x atribuindo a ele a classe mais comum entre os rótulos dos pontos em $\mathcal{V}_k(x)$, ou seja:

$$h(x) := \operatorname{argmax}_{y \in \mathcal{Y}} \sum_{\{(x_i, y_i) \in \mathcal{V}_k(x)\}} \mathbb{I}_{\{y_i = y\}} \quad (1)$$

(De forma simplificada, o rótulo mais comum dentro do conjunto de vizinhos é o rótulo atribuído ao ponto x)

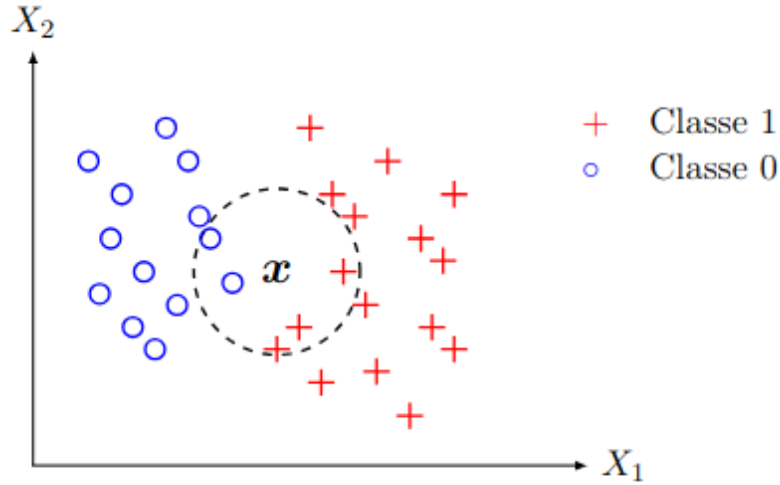


Figura 1: Exemplo de classificação usando o método k-NN. O ponto x é o ponto a ser classificado, os pontos azuis e vermelhos são os pontos do conjunto de treinamento, e as linhas tracejadas indicam as fronteiras de decisão do modelo. Aqui, se $k = 5$, ele vai classificar como **Classe 1** (vermelho)

1.3 Regressão

O k-NN pode também ser empregado em problemas de regressão. Para isso, precisamos de uma forma de combinar as saídas em $\mathcal{V}_{k(\cdot)}$. Uma das estratégias mais comuns consiste em computar a média ponderada pelo inverso da distância:

$$h(x) := \frac{1}{Z} \sum_{(x', y') \in \mathcal{V}_k(x)} \frac{y'}{d(x, x')} \quad Z := \sum_{(x', y') \in \mathcal{V}_k(x)} \frac{1}{d(x, x')} \quad (2)$$

permitindo que pontos mais próximos a x exerçam maior influência no cômputo da predição $h(x)$. Ideia semelhante pode também ser aplicada a classificação.

Observe que o algoritmo k-NN não necessita de treinamento, ou equivalentemente, o treinamento consiste em simplesmente armazenar o conjunto de dados $\mathcal{D}$. Por conta disso, k-NN é dito ser uma abordagem de lazy learning (Atkeson et al., 1997)

1.4 Qual distância escolher?

Até então, descrevemos o k-NN sem especificar exatamente o formato da função de distância d . No entanto, a escolha de uma distância apropriada pode ser crítica para o sucesso do método. Por exemplo, se os dados de entrada estão dispostos na superfície do globo terrestre, gostaríamos de usar uma distância que considere a curvatura da terra (e.g., a distância esférica).

No entanto, raramente temos esse tipo de conhecimento sobre $\mathcal{X}$ e as escolhas mais comuns para d incluem casos particulares da distância de Minkowski:

$$d_p(x, z) = \|x - z\|_p = \left(\sum_{i=1}^D |x_i - z_i|^p \right)^{\frac{1}{p}} \quad (3)$$

que para valores de $p = 1$ chama-se distância quarteirão ou Manhattan; $p = 2$ resulta na distância euclidiana; e $p \rightarrow \infty$ retorna o máximo da diferença entre as componentes dos vetores. A notação $\|\cdot\|_p$ é também chamada de norma L^p de um vetor.

Uma possível deficiência de normas L^p é que elas não incorporam nenhuma informação sobre a distribuição $\mathbb{P}_x : \mathcal{X} \rightarrow \mathbb{R}^+ \cup \{0\}$ (Que é a distribuição que os vetores x foram extraídos) — além

do fato do suporte ser subconjunto dos reais. Por exemplo, se uma componente x_i tiver escala muito maior às demais $x_{j \neq i}$, ela pode dominar o cálculo da distância, ofuscando diferenças nas demais componentes $x_{j \neq i}$. Além disso, L^p são agnósticas a correlações entre componentes de $x \sim \mathbb{P}_x$ (Quando falamos em relação, dizemos da relação entre os componentes de um x . Por exemplo, digamos que $x_i = (x_{1i}, x_{2i}, \dots, x_{Di})$ então a feature 2 e 3 são altura e peso respectivamente, sabemos que quando altura cresce, peso tende a crescer, mas a distância de Minkowski não captura essa relação). Uma alternativa para cobrir esses problema é utilizar a distância de Mahalanobis:

$$d_M(x, z) = \sqrt{(x - z)^T \Sigma^{-1} (x - z)} \quad (4)$$

em que Σ é a matriz de covariância dos dados de treinamento. Uma escolha típica para Σ é a matriz de covariância amostral, não viesada:

$$\Sigma = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})(x_i - \bar{x})^T \quad (5)$$

onde $\bar{x} := \frac{1}{N} \sum_{i=1}^N x_i$ é o vetor de médias amostrais. Observe que quando Σ é igual à matriz identidade, temos a distância euclidiana. Mas, afinal, qual distância utilizar? De modo geral, a menos que tenhamos profundo conhecimento sobre a geometria de $\mathcal{X}$, é impossível dar uma resposta direta. O melhor que podemos fazer é testar opções diferentes.

1.4.1 Normalização para média zero e variância um

Subtrair a média $\bar{x} := \frac{1}{N} \sum_{i=1}^N x_i$ de cada vetor $x_1, \dots, x_N$ e, subsequentemente, multiplicá-los pela inversa da matriz diagonal C com entradas:

$$C_{jj} = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_{ij} - \bar{x}_j)^2} \quad (6)$$

é um procedimento comum em ML, sendo geralmente chamado de normalização ou padronização (standardization). Aplicar k-NN com $d(x, z) = \|x - z\|_2$ em dados transformados dessa maneira equivale a aplicar k-NN nos dados originais usando a distância de Mahalanobis com $\Sigma^{-1} = C^{-2}$

1.4.2 Similaridade Cosseno

As distâncias estudadas até aqui são consideradas medidas de dissimilaridade (Qualidade ou estado do que é diferente, desigual ou heterogêneo) entre vetores. De modo análogo, podemos definir a vizinhança de um ponto em termos de medidas de similaridade. Uma importante medida de similaridade entre dois vetores quaisquer x e z é dada pelo cosseno do ângulo γ entre eles:

$$\cos(\gamma) = \frac{x^T z}{\|x\|_2 \|z\|_2} \quad (7)$$

A similaridade cosseno é particularmente útil quando estamos interessados na orientação, e não na magnitude, dos vetores. Ela tem sido bastante utilizada em aplicações que envolvem dados textuais (Manning & Schütze, 1999). Note também que $\cos(\gamma)$ pode ser escrito como uma função do tipo $k(x, z) = \Phi(x)^T \Phi(z)$, i.e., como uma generalização do produto interno entre x e z . Medidas de similaridade que podem ser descritas dessa forma são chamadas funções de kernel.

1.4.3 Aprendendo Métricas

Além de usar distâncias clássicas, como as L^p e a de Mahalanobis, é possível aprender métrica (ou pseudo-métrica) de distância de modo supervisionado, com base na taxa de classificação. Existe uma área de pesquisa em ML conhecida como aprendizado de métrica (metric learning)

que se dedica a essa finalidade. Nesse nicho, um dos métodos mais comuns é o chamado large margin nearest neighbor (Weinberger et al., 2006).

1.5 Maldição da Dimensionalidade

A expressão maldição da dimensionalidade foi introduzida por Bellman (1957) e é comumente usada para descrever problemas causados pelo aumento exponencial do volume associado em função da dimensionalidade em espaços euclidianos. No caso do k-NN, esse aumento implica na esparsidade dos exemplos de treino, fazendo com que os k-vizinhos que procuramos estejam muito distantes.

Para ilustrar tal efeito, suponha que a distribuição $\mathbb{P}_x$ sobre os vetores de entrada $x_1, \dots, x_N$ seja uniforme sobre uma hiperbola D -dimensional S_D centrada na origem e com raio unitário (Ou seja, todo ponto dentro dessa bola é uniformemente provável de ser escolhida para ser um vetor de entrada). Suponha também que queremos classificar o vetor de origem $z = (0, \dots, 0)^T$. Defina r como o raio da hiperbola $S'_D \subseteq S_D$ que **contém os k vizinhos mais próximos de z** . Em esperança, o quão grande devemos esperar que r seja? Antes, é intuitivo notar que $\mathbb{P}(x_i \in S'_D)$ é a razão dos volumes de S'_D e S_D , ou seja:

$$\mathbb{P}(x_i \in S'_D) = \mathbb{E}_{x_i \sim \mathbb{P}_x} [\mathbb{I}_{x_i \in S'_D}] = \frac{\pi^{\frac{D}{2}} r^D}{\pi^{\frac{D}{2}} 1^D} = r^D \quad (8)$$

Segue então que o número esperado de amostra, dentre as N que possuímos, dentro de S'_D é:

$$\sum_{i=1}^N \mathbb{P}(x_i \in S'_D) = N r^D \quad (9)$$

Então, para que tenhamos, em esperança, k vizinhos dentro de S'_D , devemos escolher r tal que $r = \left(\frac{k}{N}\right)^{\frac{1}{D}}$, e a medida que D cresce, temos:

$$\lim_{D \rightarrow \infty} r = \lim_{D \rightarrow \infty} \left(\frac{k}{N}\right)^{\frac{1}{D}} = 1 \quad (10)$$

Ou seja, quanto maior é a dimensão, maior é o raio de S'_D , o que mostra que a propriedade de “vizinhos próximos tem propriedades parecidas” é quebrada em altas dimensões, o que é um grande problema para o método k-NN.

1.5.1 Manifolds de baixa dimensão.

Na prática, não é incomum ver k-NN sendo utilizado em espaços de alta dimensão, como de imagens, e atingindo boas taxas de acurácia. Uma explicação para esse fenômeno é que os dados não estão uniformemente distribuídos e, na verdade, residem em um subespaço de baixa dimensão. Por exemplo, suponha que $\mathcal{X} \subset \mathbb{R}^{256 \times 256 \times 3}$ é o espaço de imagens de tamanho 256×256 com três canais de cores — red, green, and blue (RGB) — que contém um gato. Nós esperamos que $\mathbb{P}_x$ aloque massa zero para fotos de paisagens, obras de arte, etc

Regressão Linear

Neste capítulo, vamos estudar o uso de modelos lineares para resolução de problemas de regressão. Enquanto modelos lineares são simples, eles são amplamente empregados em várias áreas de conhecimento e fornecem uma base para a construção de modelos mais expressivos, como modelos polinomiais, métodos de kernel e redes neurais de uma única camada oculta.

2.1 O Problema

Suponha que você recebe um conjunto de dados D com N pares ordenados (x_n, y_n) , no qual $x_n \in \mathbb{R}^D$ são as entradas (ou variáveis independentes), e $y_n \in \mathbb{R}$ são amostras de uma variável de saída (ou variável dependente). Suponha ainda que cada variável de saída pode ser descrita aproximadamente como uma combinação afim do seu respectivo vetor de entradas, isto é:

$$y_n = \sum_{j=1}^D \theta_j x_{nj} + \theta_0 + \varepsilon_n = \theta^T x_n + \varepsilon_n \quad (11)$$

onde $\varepsilon_n \in \mathbb{R}$ é uma variável que reflete o grau de incerteza na observação de y_n

Nota: A partir daqui, inserimos um 1 no vetor x para representar o *bias*, então vamos redefinir $x_n := (1, x_{n1}, \dots, x_{nD})^T$ e $\theta := (\theta_0, \theta_1, \dots, \theta_D)^T$, de modo que a expressão acima possa ser escrita de forma mais compacta como $y_n = \theta^T x_n + \varepsilon_n$.

Uma das maneiras mais comuns de obter uma estimativa $\hat{\theta}$ para θ é obter o aquele que minimiza a média (ou a soma) dos erros quadráticos entre as predições $\hat{y}_n := \hat{\theta}^T x_n$ e as saídas observadas y_n :

$$\hat{\theta}_{\text{LS}} := \operatorname{argmin}_{\theta \in \mathbb{R}^{D+1}} \left\{ \ell(\theta) := \frac{1}{N} \sum_{n=1}^N (y_n - \theta^T x_n)^2 \right\} \quad (12)$$

Esse método de otimização é chamado de **mínimos quadrados ordinários**. Podemos escrever a função de custo ℓ de forma matricial como:

$$\begin{aligned} \ell(\theta) &= \frac{1}{N} \|y - X\theta\|_2^2 \\ &= \frac{1}{N} (y - X\theta)^T (y - X\theta) \\ &= \frac{1}{N} (y^T y - 2\theta^T X^T y + \theta^T X^T X \theta) \end{aligned} \quad (13)$$

Note que, quando $X^T X$ é positiva definida, a função de custo ℓ é estritamente convexa, o que garante a existência de um único mínimo global. Podemos encontrar uma solução analítica para $\hat{\theta}_{\text{LS}}$ derivando ℓ e igualando a zero:

$$\begin{aligned} \nabla_{\theta} \ell(\theta) &= \frac{1}{N} (-2X^T y + 2X^T X \theta) = 0 \\ \Rightarrow \hat{\theta}_{\text{LS}} &= (X^T X)^{-1} X^T y \end{aligned} \quad (14)$$

E se $X^T X$ não for positiva definida?: Considere o caso em que $N > D$. Para que a inversa exista, precisamos que todas as colunas de X , nossas variáveis preditoras, sejam linearmente independentes. Caso contrário, não podemos calcular a inversa na Equação (14). No entanto, essas variáveis não agregam nenhuma informação ao nosso modelo de regressão, e podemos pré-processar os dados de maneira a remover atributos redundantes. Uma outra alternativa, é substituir a inversa de $X^T X$ pela inversa de $X^T X + \alpha I$, para algum pequeno escalar α positivo. É possível provar que $X^T X + \alpha I$ sempre possui inversa (verifique!)

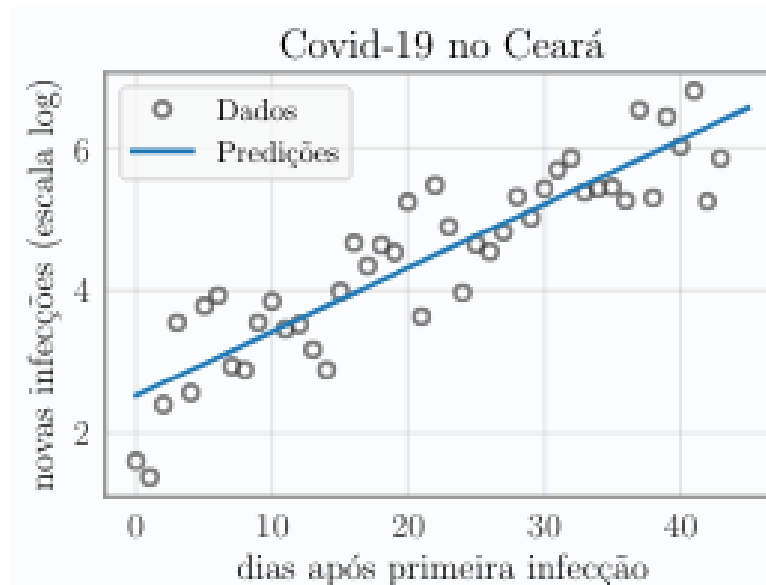
Casos $N = D$ e $N < D$: No caso em que o número de dados é igual ao número atributos ($N = D$), existe uma solução ótima única, com $\ell = 0$, se a matriz X possuir inversa. Nesses casos, a função linear resultante, com $\hat{\theta}_{LS} = X^{-1}y$, intersecta todos as observações D .

Quando $N < D$, no caso mais geral, o problema admite infinitas soluções com $\ell = 0$. Quando as linhas de X são linearmente independentes — posto(X) = N — uma opção comum é achar a solução que possui a menor norma L2, dada por $\hat{\theta}_{MN} = X^T(XX^T)^{-1}y$

Em bancos de dados massivos: Quando N é um grande número, armazenar a matriz X nas camadas de memória mais velozes de um computador se torna impraticável e, portanto, calcular $\hat{\theta}_{LS}$ usando a Equação (14) não é computacionalmente viável. Nesse caso, podemos utilizar SGD para minimizar ℓ . Para tal, note que pode-se escrever $\ell(\theta) = \sum \ell_n(\theta)$, no qual definimos $\ell_n(\theta) := \frac{1}{N}(y_n - \theta^T x_n)^2$

Exemplo (Regressão Linear Simples): No ano 2020, o mundo foi tomado por uma pandemia do vírus Sars-Cov-2, causando milhões de infecções e centenas de milhares de mortos. No dia 29 de abril, a infecção ainda não havia atingido seu estado crítico no Ceará.

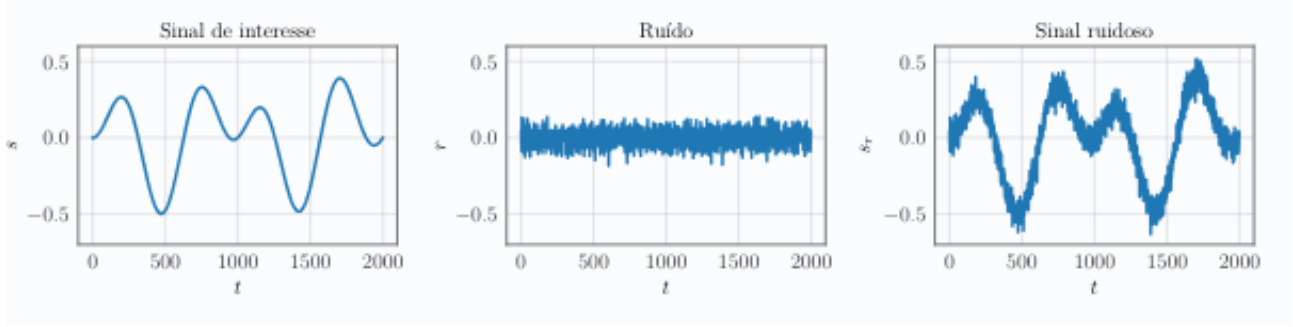
Neste exemplo, utilizamos um modelo linear para descrever o logaritmo do número de novas infecções diárias no Ceará como uma função do número de dias que se passaram desde a data na qual o primeiro caso de infecção foi reportado. Uma das utilidades de um modelo como esse é produzir estimativas para o número de novos infectados nos próximos dias, o que pode auxiliar epidemiologistas e gestores públicos em seus processos de tomada de decisão.



Vale a pena ressaltar que em geral a dinâmica de contágios em uma epidemia possui um ponto de inflexão no qual número de novos casos começa a diminuir. Portanto, um modelo linear como esse não descreve todo o processo epidemiológico e seu uso deve ser validado por um especialista.

Exemplo (Antiruído): O método de mínimos quadrados pode ser utilizado para remoção de ruído (denoising). Suponha que você recebe um sinal digital ruidoso $s_r(t)$ em que t denota um instante de tempo discreto e $t = 1, \dots, T$. Estamos interessados em encontrar a versão não ruidosa $s(t)$ de $s_r(t)$.

Um sinal ruidoso $s_r(t)$ pode ser descrito como um sinal suave $s(t)$ adicionado de um ruído de alta frequência $r(t)$:



Queremos encontrar um sinal s que seja:

- Similar ao sinal ruidoso
- Suave (a diferença entre os valores do sinal em instantes sucessivos seja pequena)

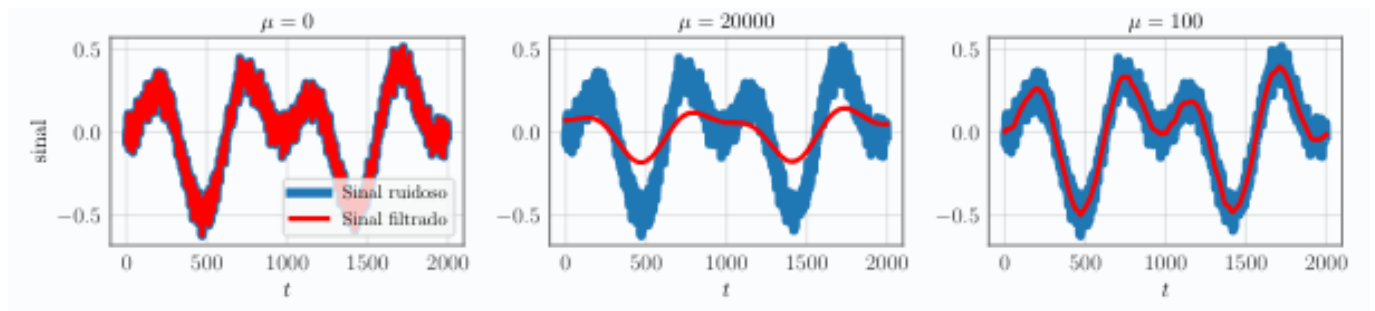
Com essas propriedades em mente, podemos propor uma função custo a se minimizar da forma:

$$\min_{s \in \mathbb{R}^T} \underbrace{\|s - s_r\|^2}_{\text{similar ao sinal ruidoso}} + \underbrace{\mu \sum_{t=1}^{T-1} (s(t+1) - s(t))^2}_{\text{suavidade}} \quad (15)$$

onde s é a representação vetorial do sinal $s(t)$. O termo μ controla o peso que queremos dar a propriedade da suavidade. Essa função custo pode ser colocada na forma $\min_s \|y - Xs\|^2$ escolhendo:

$$X = \begin{pmatrix} I_{T \times T} \\ \sqrt{\mu} D_{T-1 \times T} \end{pmatrix}, D = \begin{pmatrix} -1 & 1 & 0 & \dots & 0 \\ 0 & -1 & 1 & \dots & 0 \\ \vdots & 0 & -1 & 1 & 0 \\ 0 & 0 & \dots & -1 & 1 \end{pmatrix}, y = \begin{pmatrix} s_r \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \in \mathbb{R}^{2T-1} \quad (16)$$

Nesse formato, o valor ótimo s^* para o sinal s é obtido com a solução de mínimos quadrados $s^* = (X^T X)^{-1} X^T y$. As figuras abaixo mostram a solução ótima para valores de $\mu \in \{0, 100, 20000\}$. Para $\mu = 0$, a solução ótima é o próprio sinal ruidoso. Com $\mu = 20000$, o sinal fica muito suave, tendendo a um sinal constante. Finalmente, para $\mu = 100$, temos um sinal filtrado com eliminação da componente de ruído.



2.2 Perspectiva Probabilística

Podemos obter o mesmo estimador de mínimos quadrados se assumirmos o seguinte modelo observacional para cada resposta dado sua respectiva entrada:

$$y_n | x_n \sim N(\theta^T x_n, \sigma^2) \forall n = 1, \dots, N \quad (17)$$

podemos então procurar o estimador de máxima verossimilhança $\hat{\theta}_{\text{ML}}$ para θ :

$$\Rightarrow f_n(y|x, \theta) = \prod_{n=1}^N f(y_n|x_n, \theta) = \left(\frac{1}{(\sqrt{2\pi\sigma^2})^N} \right) \exp \left(- \sum_{n=1}^N \frac{(y_n - \theta^T x_n)^2}{2\sigma^2} \right) \quad (18)$$

$$\Rightarrow \log f_n(y|x, \theta) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - \theta^T x_n)^2 \quad (19)$$

$$\Rightarrow \hat{\theta}_{\text{ML}} = \operatorname{argmax}_{\theta \in \mathbb{R}^{D+1}} \log f_n(y|x, \theta) = \operatorname{argmin}_{\theta \in \mathbb{R}^{D+1}} \underbrace{\sum_{n=1}^N (y_n - \theta^T x_n)^2}_{N \cdot \ell(\theta)} \quad (20)$$

ou seja, a estimativa que maximiza a verossimilhança do modelo descrito pela equação (17) é equivalente à estimativa obtida minimizando a média dos erros quadrados. Mais importante, note que, quando fazemos regressão linear, estamos parametrizando um parâmetro (a média) do nosso modelo observacional como uma transformação linear do vetor de entrada.

2.3 Modelo com expansão de base

O método de mínimos quadrados aplicado a modelos lineares é atraente por sua simplicidade e pelo fato de admitir uma solução ótima analítica. No entanto, a consideração que a relação entre as variáveis de entrada e de saída é linear pode não ser válida em diversos problemas. Um das formas de combinar a vantagem de termos uma solução ótima analítica com um modelo mais geral e flexível do que o linear é utilizar uma transformação não-linear das variáveis de entrada.

Nessa abordagem, de forma geral, o primeiro passo consiste em transformar as variáveis de entrada x através de uma função $\Phi : \mathbb{R}^D \rightarrow \mathbb{R}^M$, em que normalmente M é maior que D . Em seguida, aplicamos uma transformação linear sobre as variáveis transformadas para obtermos as predições $\hat{y}$ para as variáveis de saída y . Ou seja, considere as variáveis transformadas $z := \Phi(x)$, o modelo preditivo é dado por:

$$\hat{y} = \theta^T z = \theta^T \Phi(x) \quad (21)$$

onde o vetor $\theta \in \mathbb{R}^M$ denota os parâmetros do modelo

Note que o modelo é não linear com relação às entradas originais x , mas é linear no espaço das variáveis z . Quando a transformação Φ é fixa, sem parâmetros a serem aprendidos, o modelo é dito ser linear nos parâmetros. Nesses casos, a solução de mínimos quadrados é obtida simplesmente substituindo a matriz original de regressores X por uma matriz $Z = [z_1, \dots, z_N]^T$ de entradas transformadas na equação (14):

$$\hat{\theta}_{\text{LS}} = (Z^T Z)^{-1} Z^T y \quad (22)$$

A transformação Φ atua como um pré-processamento das entradas $x_1, \dots, x_N$. A seguir, estudaremos algumas das escolhas mais comuns para Φ

Por que $M > D$? As variáveis de entrada x representam atributos de um objeto sob o qual queremos realizar predições. Em geral quando aplicamos a transformação não-linear Φ queremos encontrar novos atributos z que permitam ao modelo linear ser preciso. Dessa forma, é natural trabalharmos em espaços com dimensões maiores, aumentando a chance de encontrarmos atributos relevantes. Vale ressaltar que isso não constitui uma regra. Conforme

veremos adiante, o aumento do valor M pode gerar problemas, especialmente quando temos poucas amostras proporcionalmente a M

2.3.1 Polinômios

Funções de expansão de base podem ser utilizadas para construir modelos polinômiais. Para entradas e saídas escalares, podemos descrever um modelo de regressão polinomial de grau dois como:

$$\hat{y} = \theta_3 + \theta_2 x + \theta_1 x^2 = \theta^T \Phi(x) = \theta^T z \quad (23)$$

onde $z = \Phi(x)$ é dado por:

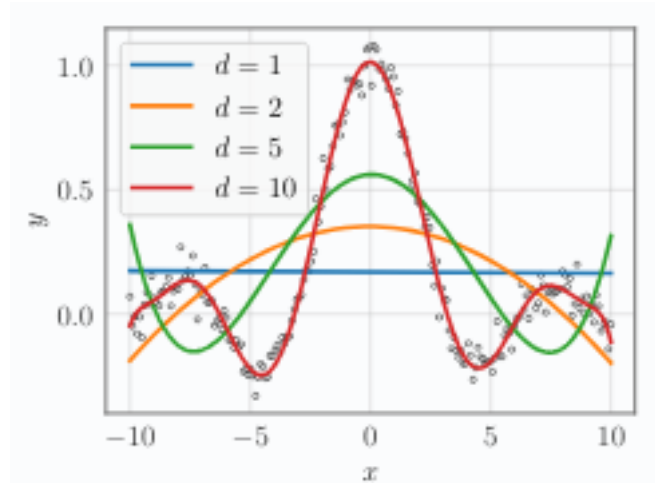
$$z = \Phi(x) = \begin{pmatrix} x^2 \\ x \\ 1 \end{pmatrix} \quad (24)$$

O mesmo pode ser feito para entradas multivariadas (A função Φ fica um pouco mais complexa) e qualquer expansão de grau polinomial finito. Por exemplo, para entradas bidimensionais, obtemos um modelo de grau 2 se:

$$\Phi(x) = \begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1 x_2 \\ x_1 \\ x_2 \\ 1 \end{pmatrix} \quad (25)$$

Exemplo (Regressão Polinomial em funções não-lineares): Considere o problema de regressão univariada ($D = 1$) em que as entradas pertencem ao intervalo $[-10, 10]$ e a função alvo é dada por $f(x) = \sin(x)/x$, também conhecida como função *sinc*. Além disso, utilizamos um conjunto de dados com 200 pares de entrada-saída (x_i, y_i) , em que as saídas estão corrompidas por um ruído aditivo gaussiano, ou seja, $y = f(x) + \varepsilon$ com $\varepsilon \sim N(0, 0.05)$. Nesse exemplo, empregamos modelos polinomiais com grau $d \in \{1, 2, 5, 10\}$.

A Figura abaixo mostra as previsões obtidas com os diferentes modelos. Observe que, para $d = 1$, temos o modelo de regressão linear básico que estudamos anteriormente, e a aproximação consiste em uma reta. Note que, à medida que aumentamos o grau do polinômio, o modelo se torna mais flexível, conseguindo aproximar melhor os dados (representados por pequenos círculos pretos), e portanto o melhor modelo possui $d = 10$ (curva em vermelho)



2.3.2 Funções de base radiais

Vamos agora estudar um novo formato para a transformação Φ . De forma simples, ele consiste em escolher M pontos $c_1, c_2, \dots, c_M$ do $\mathbb{R}^D$, também chamado de centros ou protótipos, e então criar o vetor de regressores $z = \Phi(x)$ combinando funções radiais em torno de cada um dos centros.

Definição 2.3.2.1 (Função radial no $\mathbb{R}^D$): Dada uma métrica $\|\cdot\|$ no $\mathbb{R}^D$ e $c \in \mathbb{R}^D$, dizemos que $f_c : \mathbb{R}^D \rightarrow \mathbb{R}$ é radial se existe uma função $f : [0, \infty) \rightarrow \mathbb{R}$ tal que $f_c(x) = f(\|x - c\|)$

Podemos então construir uma transformação Φ que utiliza M funções radiais da forma:

$$\Phi(x) = \begin{pmatrix} f_{c_1}(x) \\ f_{c_2}(x) \\ \vdots \\ f_{c_M}(x) \end{pmatrix} = \begin{pmatrix} f(\|x - c_1\|) \\ f(\|x - c_2\|) \\ \vdots \\ f(\|x - c_M\|) \end{pmatrix} \quad (26)$$

Quando as funções $f_{c_1}(x), \dots, f_{c_M}(x)$ são linearmente independentes, e a matriz

$$\begin{pmatrix} f_{c_1}(c_1) & f_{c_2}(c_1) & \dots & f_{c_M}(c_1) \\ f_{c_1}(c_2) & f_{c_2}(c_2) & \dots & f_{c_M}(c_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_{c_1}(c_M) & f_{c_2}(c_M) & \dots & f_{c_M}(c_M) \end{pmatrix} \quad (27)$$

é não-singular, essas funções são chamadas de funções de base radiais (radial basis functions, RBFs). modelo de regressão linear com transformações através de funções de base radiais constitui uma classe de rede neurais chamada redes RBF (Broomhead & Lowe, 1988)

A completa definição da transformação Φ envolve duas escolhas: a localização dos centros $c_1, \dots, c_M$ e a função de base radial.

Escolhendo os centros. Um das formas mais simples de escolher M protótipos consiste em selecionar aleatoriamente entradas x_i do próprio conjunto de dados. Nessa abordagem, o conjunto de centros é um subconjunto $\{c_1, \dots, c_M\} \subseteq \{x_1, \dots, x_N\}$ qualquer de tamanho M .

No entanto, a estratégia mais comum e que se tornou padrão consiste em selecionar centros de forma a capturar a densidade dos vetores de entrada. Para isso, normalmente empregamos métodos de análise de agrupamentos (clustering), tais como o k-médias (Lloyd, 1982). A discussão sobre o impacto da escolha dos centros está fora do escopo deste material

Escolhendo a função de base radial. Existem diversas escolhas possíveis para f_{c_i} , algumas das mais notórias são:

1. Gaussiana:

$$f_{c_i}(x) = e^{-\gamma \|x - c_i\|_2^2} \quad (28)$$

onde $\gamma > 0$ é uma constante;

2. Multi-quadrática:

$$f_{c_i}(x) = \sqrt{1 + \varepsilon \|x - c_i\|_2^2} \quad (29)$$

onde $\varepsilon > 0$ é uma constante.

Redes RBF e interpolação: As redes RBF foram originalmente propostas para resolver problemas de interpolação: Dado um conjunto de N pares entrada saída (x_i, y_i) , queremos encontrar uma função g tal que

$$g(x_i) = y_i, \quad i = 1, \dots, N \quad (30)$$

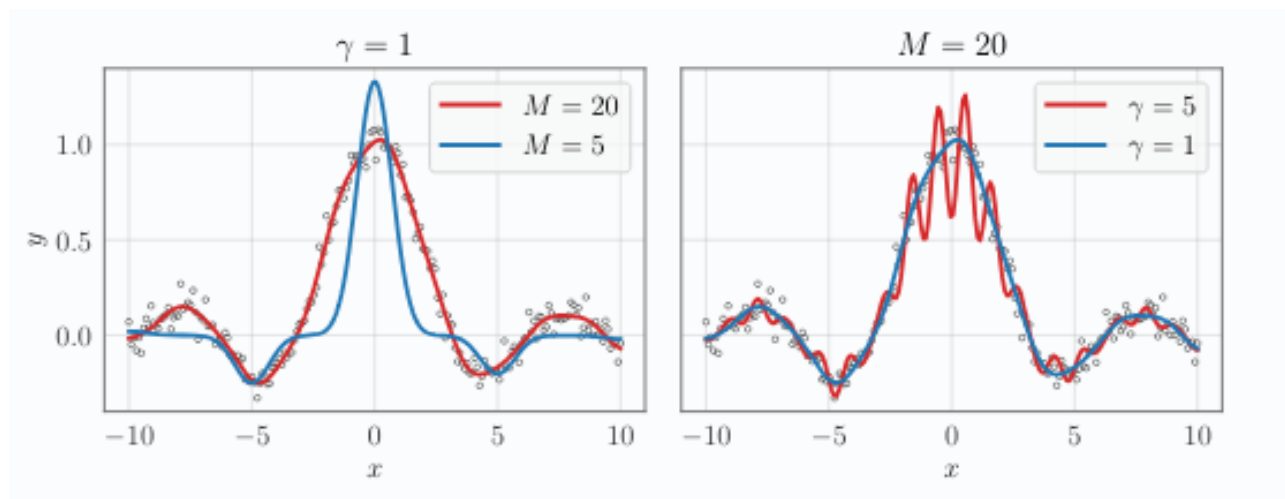
Escolhendo g tal que $g(x) = \theta^T \Phi(x) = \theta^T z$ com N funções de base radiais e $c_i = x_i$ para todo $i = 1, \dots, N$, a matriz de regressores $Z : Z_{ij} = f_{c_j}(x_i)$ é quadrada e a condição de interpolação equivale ao sistema linear:

$$Z\theta = y \text{ com } Z = \begin{pmatrix} f_{c_1}(x_1) & f_{c_2}(x_1) & \dots & f_{c_N}(x_1) \\ f_{c_1}(x_2) & f_{c_2}(x_2) & \dots & f_{c_N}(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_{c_1}(x_N) & f_{c_2}(x_N) & \dots & f_{c_N}(x_N) \end{pmatrix} \quad (31)$$

Dessa forma, obtemos um interpolador com a rede RBF desde que a matriz Z seja não-singular. Micchelli (1986) mostrou que matrizes Z formadas usando tanto funções radiais gaussianas quanto multi-quadráticas são não-singulares (possuem inversa), e a única condição para isso é que os centros (ou equivalentemente as entradas) sejam distintos. Como consequência, no caso em que $M < N$ e os centros são um subconjunto das entradas, a matriz $Z^T Z$ utilizada na solução dos mínimos quadrados possui inversa.

Exemplo (Redes RBF): Este exemplo ilustra o impacto do número de funções de base na aproximação realizada por redes RBF com função radial Gaussiana. Para isso, vamos utilizar novamente o problema de regressão com função alvo *sinc*, com exatamente a mesma configuração descrita no último exemplo. Os centros das RBFs foram selecionados de forma igualmente espaçados no intervalo $[-10, 10]$.

A Figura abaixo (lado esquerdo) mostra a aproximação obtida usando uma rede RBF com $M \in \{5, 20\}$ e $\gamma = 1$. Note que o modelo com $k = 20$ aproxima melhor os dados. O aumento no número de funções de base aumenta a flexibilidade do modelo em se ajustar aos dados. Na Figura à direita, percebemos que o aumento no valor de γ produz funções radiais com variância (ou largura de banda) pequena, resultando em um aspecto oscilatório da curva de aproximação



Regressão Logística

Uma das maneiras mais naturais de criar um modelo de regressão consiste em escolher um modelo observacional para a variável de resposta y cujos parâmetros dependam diretamente do seu respectivo vetor de entradas x . Por exemplo, na capítulo anterior, vimos que minimizar o MSE é equivalente a admitir uma verossimilhança da forma $y|x \sim N(\theta^T x, \sigma^2)$, na qual o parâmetro de média é uma função linear de x . Note também que essa escolha implica que y pode tomar valores arbitrários em $\mathbb{R}$, já que esse é o suporte da distribuição normal (i.e., região com densidade maior que zero).

Se y é uma variável binária (0/1), a escolha mais comum é utilizar uma distribuição Bernoulli com parâmetro $r = g(x)$. Em outras palavras, falamos que y assume valor 1 com probabilidade r e 0 com probabilidade $1 - r$. Adotando esse modelo observacional para $y|x$, resta-nos definir a função g para completar nosso modelo de regressão. Para tal, iremos calcular uma função linear de x , como anteriormente, mas aplicaremos uma função que mapeie o valor resultante (comumente conhecido como logit) para $[0, 1]$, gerando valores válidos para a probabilidade r . Mais especificamente, usamos a função sigmoide $\sigma(t) = (1 + e^{-t})^{-1}$ para definir a probabilidade de $y|x$ como

$$p(y|x) = \text{Bern}(y|\sigma(\theta^T x)) = \sigma(\theta^T x)^y (1 - \sigma(\theta^T x))^{1-y} \quad (32)$$

Propriedades da função sigmoide

$$\rightarrow t < t' \Rightarrow \sigma(t) < \sigma(t') \quad (33)$$

$$\lim_{t \rightarrow \infty} \sigma(t) = 1 \quad \lim_{t \rightarrow -\infty} \sigma(t) = 0 \quad (34)$$

$$\sigma(-t) = 1 - \sigma(t) \quad (35)$$

$$\frac{d}{dt} \sigma(t) = \sigma(t)(1 - \sigma(t)) = \sigma(t)\sigma(-t) \quad (36)$$

$$\sigma(t) = \frac{1}{2} + \frac{1}{2} \tanh\left(\frac{t}{2}\right) \quad (37)$$

$$\int \sigma(t) dt = \log(\sigma(-t)) + C \quad (38)$$

Dado um conjunto de treinamento D com N exemplos de treinamento (x_n, y_n) , podemos então definir a função de verossimilhança $\mathcal{L}$ como

$$\mathcal{L}(\theta) = \prod_{i=1}^N p(y_i|x_i) = \prod_{i=1}^N \sigma(\theta^T x_i)^{y_i} (1 - \sigma(\theta^T x_i))^{1-y_i} \quad (39)$$

e agora podemos encontrar o estimador de máxima verossimilhança $\hat{\theta}$ para θ :

$$\begin{aligned} \hat{\theta} &= \operatorname{argmax}_{\theta \in \mathbb{R}^{D+1}} \mathcal{L}(\theta) = \operatorname{argmin}_{\theta \in \mathbb{R}^{D+1}} -\log \mathcal{L}(\theta) \\ &= \operatorname{argmin}_{\theta \in \mathbb{R}^{D+1}} -\sum_{i=1}^N [y_i \log \sigma(\theta^T x_i) + (1 - y_i) \log(1 - \sigma(\theta^T x_i))] \\ &= \operatorname{argmin}_{\theta \in \mathbb{R}^{D+1}} -\left\{ \sum_{\substack{i=1 \\ y_i=1}}^N \log \sigma(\theta^T x_i) - \sum_{\substack{i=1 \\ y_i=0}}^N \log \sigma(-\theta^T x_i) \right\} \end{aligned} \quad (40)$$

Similar ao MSE, que minimizamos no capítulo passado, a função objetivo $-\log(\mathcal{L})$ é uma convexa. No entanto, não possuímos uma solução analítica para $\hat{\theta}$ e, portanto, precisamos utilizar algum método de otimização numérica, como o SGD que já conhecemos. Para fins de implementação, podemos definir a variável $y'_i = 2y_i - 1$. Com isso, conseguimos escrever o gradiente $\nabla \theta \mathcal{L}_i(\theta)$ da log verossimilhança para o i -ésimo exemplo de treinamento $\mathcal{L}_i$ como:

$$\begin{aligned}\nabla_{\theta} \mathcal{L}_i(\theta) &= \nabla_{\theta} \log \sigma(y'_i \theta^T x_i) \\ &= \frac{\partial \log \sigma(y'_i \theta^T x_i)}{\partial \sigma(y'_i \theta^T x_i)} \cdot \frac{\partial \sigma(y'_i \theta^T x_i)}{\partial (y'_i \theta^T x_i)} \cdot \frac{\partial y'_i \theta^T x_i}{\partial \theta} \\ &= \sigma(-y'_i \theta^T x_i) y'_i x_i\end{aligned}\tag{41}$$

Aplicando o algoritmo gradiente descendente à função custo da regressão logística, obtemos o seguinte algoritmo:

Regressão Logística

```

1  $\theta_0 \leftarrow 0$ 
2  $y' \leftarrow 2y - 1$ 
3 for  $t = 0, 1, 2, \dots$  do
4    $g \leftarrow \sum_{i=1}^N \sigma(-y'_i \theta_t^T x_i) y'_i x_i$ 
5    $\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta g$ 
6   Verifica condição de parada
7 end for
8 return  $\theta$ 
```

Algoritmo 1: Regressão Logística

Interpretação geométrica: Uma vez que obtivemos $\hat{\theta}$, podemos estimar a probabilidade de uma nova amostra x^* pertencer à classe 1 como $\sigma(\hat{\theta}^T x^*)$ e a de pertencer à classe 0 como $1 - \sigma(\hat{\theta}^T x^*)$. Com isso em mente, se precisamos prever a classe de x^* , é razoável escolher aquela que achamos mais provável. Lembre que $\sigma(0) = 0.5$. Portanto, o plano $\hat{\theta}^T x = 0$ caracteriza os pontos $x \in \mathcal{X}$ que cremos ter probabilidade idêntica de pertencer a ambas as classes. Isso implica que todos os vetores de entrada cujo ângulo γ com o vetor normal $\hat{\theta}$ é menor que noventa graus são classificados como positivos (classe 1) — lembre que $\theta^T x = \|\hat{\theta}\|_2 \|x\|_2 \cos(\gamma)$. Os demais pontos são classificados como negativos (classe 0).

Convexidade do problema de aprendizado: Como a soma de funções convexas é também convexa, basta verificar que as $\mathcal{L}_1, \dots, \mathcal{L}_N$ são convexas para provarmos que $-\log \mathcal{L}$ também o é. Para esse fim, podemos usar o fato de que a função composta $h = g \circ f$ é convexa se f é côncava e g é convexa não-crescente. Note que $y'_i \theta^T x_i$ é tanto côncava como convexa, como é o caso de funções lineares. Em contrapartida, $-\log \sigma(t)$ é convexa não-crescente já que ela decresce com t e sua derivada $-\sigma(-t)$ é estritamente crescente

Entropia Cruzada Binária: Na comunidade de ML, é comum se referir ao logaritmo negativo da verossimilhança Bernoulli como entropia cruzada binária (binary cross entropy, BCE). De

forma geral, a entropia cruzada entre duas funções de massa/densidade p e q sobre a mesma variável aleatória z e com suportes idênticos é definida como:

$$H(p, q) := -\mathbb{E}_{z \sim p} \log q(z) \quad (42)$$

e dá-se o nome BCE para o caso especial em que p e q são distribuições Bernoulli.

Em teoria da informação, é comum interpretar $\log \frac{1}{q(z)}$ como uma medida de surpresa, i.e., do quanto observar um valor específico z contrasta com seu conhecimento prévio, representado por q . Nesse contexto, $H(p, q)$ é o valor dessa medida se os valores de z são amostrados de p (ao invés de q). Vale ressaltar que, para um p fixo, $q = p$ minimiza $H(p, q)$. Nesse caso, a quantia $H(p) := H(p, p)$ é chamada de entropia. Por sua vez, a entropia também pode ser vista como uma medida de concentração de p , atingindo seu valor máximo quando p é uma distribuição uniforme.

Para concluir que a BCE generaliza $-\log \text{Ber}(y|r)$, basta definir $p(z) = \text{Ber}(z|y)$ e tomar $q(z) = \text{Ber}(z|r)$. Com essas escolhas, obtemos:

$$\begin{aligned} H(p, q) &= -y \log q(1) - (1 - y) \log q(0) \\ &= -(y \log r + (1 - y) \log(1 - r)) \end{aligned} \quad (43)$$

o que implica que:

$$e^{-H(p, q)} = r^y (1 - r)^{1-y} = \text{Ber}(y|r) \quad (44)$$

3.1 Regressão Logística Bayesiana

Na seção anterior, discutimos como obter uma estimativa pontual $\hat{\theta}$ para θ via máxima verossimilhança (MLE). Um problema com estimativas pontuais, no entanto, é que elas não refletem nossa incerteza sobre o valor estimado. Intuitivamente, por exemplo, esperamos que estimativas feitas com grandes quantidades de dados sejam mais confiáveis que as feitas com poucos dados. Para observar que MLE não captura esse aspecto, basta notar que repetir o conjunto de dados qualquer número arbitrário de vezes não causa impacto algum em $\hat{\theta}$.

A estatística Bayesiana propõe uma saída intuitiva para esse empasse. A ideia é modelar os parâmetros θ do modelo como uma variável aleatória, usando uma distribuição a priori $p(\theta)$ e aplicar a regra de Bayes para computar a distribuição de θ condicionada nas observações D , que chamamos de posteriori:

$$p(\theta|D) = \frac{p(D|\theta)p(\theta)}{p(D)} = \frac{\mathcal{L}(\theta)p(\theta)}{\int \mathcal{L}(\theta')p(\theta') d\theta'} \quad (45)$$

Informalmente, a posteriori representa a incerteza que temos sobre o valor da variável θ . Além disso, vale ressaltar que a priori nos permite encapsular conhecimento prévio, potencialmente subjetivo, sobre θ (i.e., antes de ver os dados) e sua escolha é uma questão de modelagem estatística. Quando não possuímos informação significativa sobre os dados, é comum escolher uma distribuição de alta entropia como priori. No caso em que θ assume valores reais, e.g., poderíamos usar uma priori Gaussiana com alta variância.

No caso da regressão logística, é comum adotar uma priori Gaussiana sobre θ e a verossimilhança que usamos na seção anterior, resultando no modelo

$$\begin{aligned}\theta &\sim N(\mu, \Sigma) \\ y_n|x_n, \theta &\sim \text{Bern}(\sigma(\theta^T x_n)) \quad \forall n = 1, \dots, N\end{aligned}\tag{46}$$

cuja posteriori é dada por

$$p(\theta|D) = \frac{\prod_{n=1}^N \text{Bern}(y_n|\sigma(\theta^T x_n)) N(\theta|\mu, \Sigma)}{\int_{x \in \mathbb{R}^{D+1}} \prod_{n=1}^N \text{Bern}(y_n|\sigma(\theta'^T x_n)) N(\theta'|\mu, \Sigma)}\tag{47}$$

Notavelmente, computar a posteriori acima depende da resolução de uma integral que não possui forma fechada. Consequentemente, também não há uma forma analítica para $p(\theta|D)$. Essa dificuldade técnica não é uma raridade em modelos Bayesianos. Apesar de algumas escolhas pareadas de verossimilhança e priori resultarem em posteriores com forma analítica, esse não é o caso geral. Para driblar esse problema, usaremos métodos numéricos para aproximar a posteriori com distribuição mais simples, de forma conhecida

Prevendo a label y^* para um novo input x^* : Suponha que conseguimos computar a posteriori $p(\theta|D)$, como podemos obter uma distribuição para $p(y^*|x^*)$? Quando estavamos usando uma estimativa pontual $\hat{\theta}$ (MLE), obtivemos uma distribuição sobre y^* simplesmente encaixando $\hat{\theta}$ na nossa verossimilhança, i.e., tomamos $p(y^*|x^*) \approx \text{Ber}(\sigma(\hat{\theta}^T x^*))$. No paradigma Bayesiano, levamos em consideração a incerteza sobre θ (codificada em nossa posteriori), ponderando cada valoração de θ pela sua densidade posteriori. O resultado, é o que chamamos de posteriori preditiva

$$p(y^*|x^*) = \int p(y^*|x^*, \theta) p(\theta|D) d\theta\tag{48}$$

Máxima verossimilhança e máximo a posteriori: Uma das maiores virtudes do paradigma Bayesiano é oferecer uma maneira de quantificar incerteza. No entanto, há situações nas quais computar a posteriori, mesmo que de maneira aproximada, pode se tornar computacionalmente indesejável. Nesses casos, é comum procurar o ponto que maximiza a posteriori e tomá-lo como estimativa pontual. Chama-se esse procedimento de máximo a posteriori (MAP). Mais concretamente, a estimativa $\hat{\theta}_{\text{MAP}}$ pode ser obtida como:

$$\begin{aligned}\hat{\theta}_{\text{MAP}} &= \text{argmax}_{\theta} \log p(\theta|D) \\ &= \text{argmax}_{\theta} \left\{ \log p(D|\theta) + \log p(\theta) - \log \int_{\theta} p(D|\theta) p(\theta) \right\} \\ &= \text{argmax}_{\theta} \{ \log \mathcal{L}(\theta) + \log p(\theta) \}\end{aligned}\tag{49}$$

e, portanto, pode ser interpretada como uma versão regularizada do MLE, na qual $\log p(\theta)$ penaliza regiões pouco prováveis a priori

Exemplo (Priori conjugada): Para escolhas específicas de priori e verossimilhança, a distribuição posteriori possui forma analítica. Uma instância dessas ocorre quando $p(\theta)$ é uma distribuição Beta e a verossimilhança $p(D|\theta)$ é Bernoulli. Mais concretamente, suponha que escolhemos uma priori Beta(α, β) para θ , dada por:

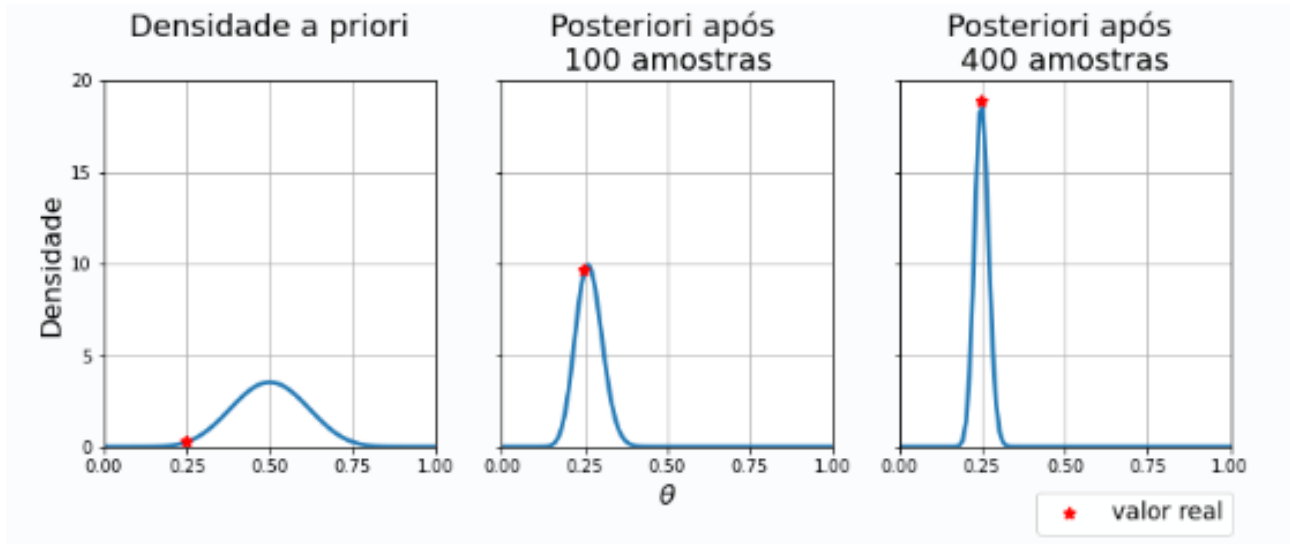
$$B(\theta|\alpha, \beta) = \left(\frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \right) \theta^{\alpha-1} (1 - \theta)^{\beta-1}\tag{50}$$

Podemos inferir, então, o seguinte sobre a posteriori do nosso modelo:

$$\begin{aligned}
p(\theta|D) &\propto \prod_{n=1}^N \text{Bern}(y_n|\theta) \text{Beta}(\theta|\alpha, \beta) \\
&\propto \prod_{n=1}^N \theta^{y_n} (1-\theta)^{1-y_n} \theta^{\alpha-1} (1-\theta)^{\beta-1} \\
&\propto \theta^{\alpha + \sum_{n=1}^N y_n - 1} (1-\theta)^{\beta + N - \sum_{n=1}^N y_n - 1} \\
&\propto \theta^{\alpha' - 1} (1-\theta)^{\beta' - 1} = \text{Beta}(\theta|\alpha', \beta')
\end{aligned} \tag{51}$$

onde $\alpha' = \alpha + \sum_{n=1}^N y_n$ e $\beta' = \beta + N - \sum_{n=1}^N y_n$.

Portanto, concluímos que nossa posteriori é uma $\text{Beta}(\alpha', \beta')$. Para ilustrar o uso da regra de Bayes, a figura abaixo mostra atualizações da posteriori derivada acima para diferentes números de amostras N . Para tal, assumimos que a distribuição geradora dos dados é $\text{Bern}(0.25)$ e usamos uma priori $\text{Beta}(10, 10)$. Veja que, à medida que vemos mais amostras, a posteriori se afunila ao redor de 0.25.



3.1.1 Aproximação de Laplace

A aproximação de Laplace é, possivelmente, a mais simples técnica de inferência Bayesiana aproximada. A ideia é construir uma aproximação simples $q(\theta)$ para a posteriori $p(\theta|D)$ usando uma expansão de Taylor de segunda ordem em $\log p(\theta|D)$ ao redor da moda m da posteriori (i.e., o ponto de máxima densidade)

$$\log p(\theta|D) \approx \log p(m|D) + (\theta - m)^T \nabla_{\theta} \log p(m|D) + \frac{1}{2} (\theta - m)^T \nabla_{\theta}^2 \log p(m|D) (\theta - m) \tag{52}$$

Note que $\log p(m|D)$ é uma constante com respeito a θ e lembre que o gradiente de uma função em sua moda, caso ela exista, é zero. Então, concluímos que:

$$\log p(\theta|D) \approx \frac{1}{2} (\theta - m)^T \mathbf{H} (\theta - m) + C \tag{53}$$

onde $\mathbf{H} = \nabla_{\theta}^2 \log p(m|D)$. Por design, construímos q tal que $\log q$ difira da expansão acima apenas por uma constante aditiva, então:

$$\log q(\theta) = \frac{1}{2} (\theta - m)^T \mathbf{H} (\theta - m) + C' \Rightarrow q(\theta) \propto \exp\left(\frac{1}{2} (\theta - m)^T \mathbf{H} (\theta - m)\right) \tag{54}$$

e como $q(\theta)$ é proporcional á uma densidade normal multivariada com média m e matriz de covariância igual á inversa de H , temos:

$$q(\theta) = N(\theta | \mu = m, \Sigma = H^{-1}), \text{ onde } m = \operatorname{argmax}_{\theta} p(\theta | D) \text{ e } H = \nabla_{\theta}^2 - \log p(m | D) \quad (55)$$

Note que o procedimento acima envolve inverter H . Como m é um mínimo local para a função $-\log p(\cdot | D)$, segue diretamente das condições de optimalidade de segunda ordem que H é PSD. Caso H não seja PD ou haja instabilidade numérica na inversão de H , uma prática comum é adicionar um pequeno valor $c > 0$ à sua diagonal.

Para aplicar o método de Laplace ao nosso modelo de regressão logística Bayesiano, podemos usar alguma variação de gradiente descendente para achar m e, assumindo $\mu = 0$ e $\Sigma = cI$ para $c > 0$, as entradas H_{ij} da Hessiana H são dadas por

$$H_{ij} = \begin{cases} \sum_{n=1}^N \sigma(\theta^T x_n) \sigma(-\theta^T x_n) x_{ni} x_{nj} & \text{se } i \neq j \\ \sum_{n=1}^N \sigma(\theta^T x_n) \sigma(-\theta^T x_n) x_{ni}^2 + c^{-1} & \text{se } i = j \end{cases} \quad (56)$$

3.2 Problemas multiclasse, classificador softmax

Nesse capítulo, nos focamos em problemas de classificação binária, em que $|\mathcal{Y}| = 2$. No entanto, é fácil generalizar as técnicas que discutimos para problemas multi-classe (i.e., $|\mathcal{Y}| > 2$). Para tal, basta substituir o nosso modelo observacional Bernoulli por uma distribuição categórica. Lembre que a Bernoulli é parametrizada por um parâmetro escalar que dita a probabilidade de cada classe. No caso da categórica, precisamos de um vetor de probabilidades, i.e., um vetor r de tamanho $L = |\mathcal{Y}|$, em que cada entrada r_l denota a probabilidade da classe l . Naturalmente, todas as entradas de r devem ser não-negativas e $\sum_{l=1}^L r_l = 1$. Resta-nos, então, expressar r como uma função de x . Para tal, podemos generalizar nosso o procedimento que usamos para regressão logística.

Primeiro, calculamos um vetor de logits z , desta vez usando uma transformação linear para cada uma das L classes

$$z = \begin{pmatrix} x^T \theta^{(1)} \\ x^T \theta^{(2)} \\ \vdots \\ x^T \theta^{(L)} \end{pmatrix} \quad (57)$$

Finalmente, aplicamos a função Softmax para transformar z em um vetor de probabilidades e obter r , que é dado por:

$$r_l = \operatorname{Softmax}(z) = \frac{e^{z_l}}{\sum_{l'=1}^L e^{z_{l'}}} \quad (58)$$

Redes Neurais

Redes Neurais são modelos de regressão linear com transformações não-lineares parametrizadas. De forma intuitiva, é como se fossemos aplicando diversas transformações não-lineares $\Phi_1, \Phi_2, \dots, \Phi_K$ em sequência até obtermos uma representação de x que seja adequada para a predição de y através de um modelo linear ou logístico. Nesse capítulo, vamos introduzir a ideia mais simples de redes neurais, generalizar para inúmeras camadas e introduzir o algoritmo de retropropagação, que é a técnica mais comum para treinar redes neurais (Além de provar ele matematicamente)

4.1 Estrutura Inicial

As redes neurais são estruturadas com base em camadas de neurônios interconectados, de forma que cada camada tem sua própria quantidade de neurônios e cada neurônio é conectado a todos os neurônios da camada seguinte.

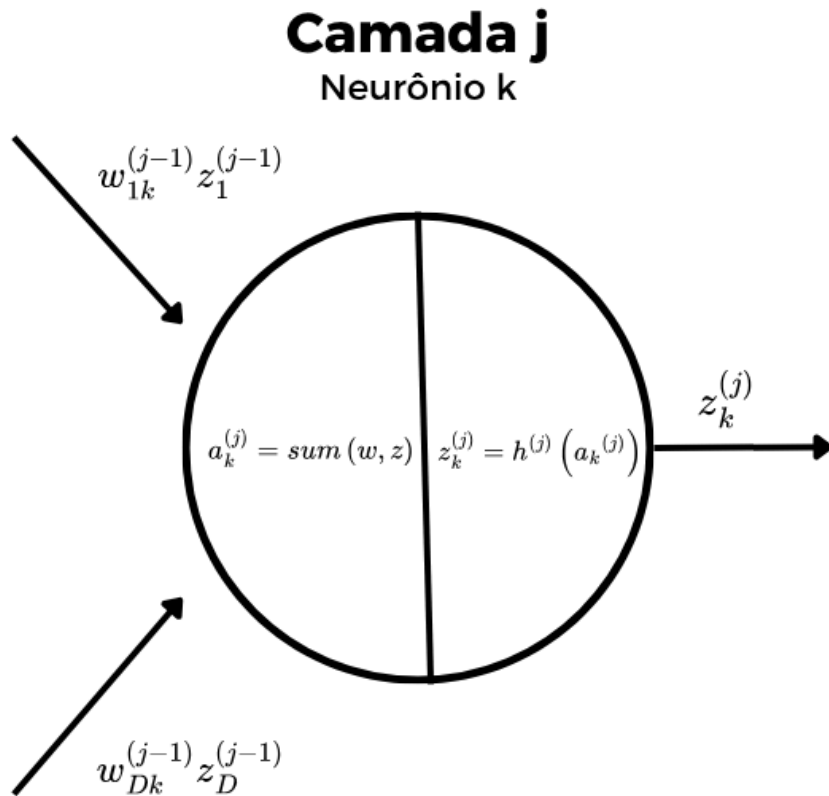


Figura 8: Representação do neurônio k da camada j . $sum(w, z) = b_k^{(j)} + \sum_{i=1}^D w_{ik}^{(j-1)} z_i^{(j-1)}$

Na primeira camada, $Z = X$. Mas antes de continuarmos vendo isso, vamos definir as camadas em si:

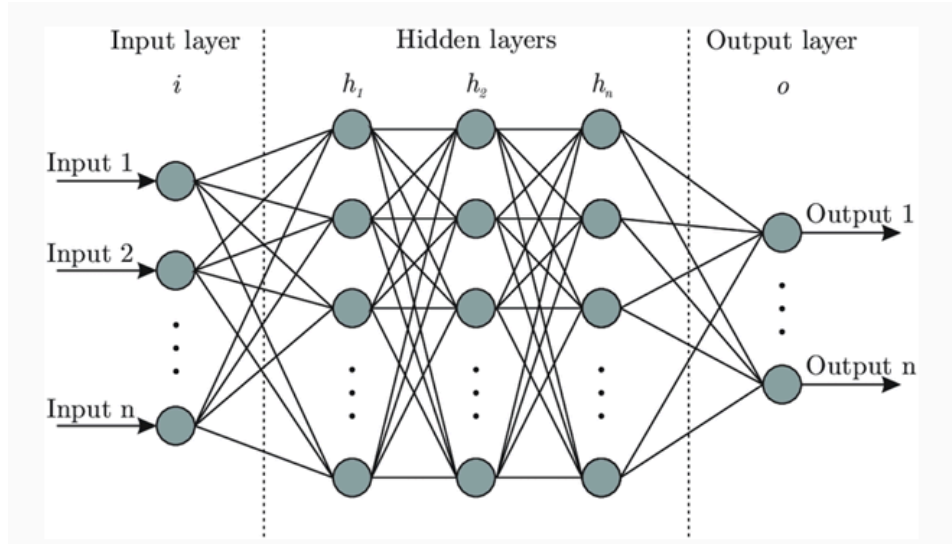


Figura 9: Camadas de entrada, escondidas e de saída

Então a estrutura de uma rede neural com K camadas escondidas é, pegar o datapoint, joga na camada de entrada, ele é processado dentro de todas as camadas e finalmente chega na camada de saída, onde é feita a predição.

4.2 Estrutura Matemática

Vamos agora formalizar a estrutura matemática das Redes Neurais. Pelo que você viu na imagem anterior, cada camada j tem um número de neurônios M_j . O número de neurônios da camada de entrada é $M_0 = D$ e o número de neurônios da camada de saída é $M_{K+1} = L$. Para cada camada $j \in \{1, \dots, K+1\}$, temos uma matriz de pesos $W^{(j)}$ de tamanho $M_{j-1} \times M_j$ e um vetor de bias $b^{(j)}$ de tamanho M_j . Podemos escrever o resultado da última camada Z^{K+1} como:

$$Z^{(3)} = h^{(3)}(h^{(2)}(h^{(1)}(XW^{(1)} + b^{(1)}\mathbf{1}^T)W^{(2)} + b^{(2)}\mathbf{1}^T)W^{(3)} + b^{(3)}\mathbf{1}^T) \quad (59)$$

(Exemplo com 3 camadas, 1 de entrada, 1 escondida e 1 de saída)

De forma que a i -ésima coluna de $W^{(j)}$ representa os pesos de todas as conexões que chegam no neurônio i da camada j e a i -ésima entrada de $b^{(j)}$ é o bias do neurônio i da camada j . A função $h^{(j)}$ é a função de ativação da camada j , que é aplicada elemento a elemento

4.3 Não-linearidade

É importante ressaltar que, se as funções de ativação $h^{(1)}, \dots, h^{(K)}$ forem todas lineares, então a rede neural é equivalente a um modelo de regressão linear sem camadas escondidas, pois, se h é uma função linear:

$$h(X) = AX \quad (60)$$

para algum X

4.4 Treinamento

O treinamento da rede neural é dividido em 3 passos:

1. **Forward Pass:** Não é nada além de passar os dados pela rede neural e armazenar os resultados de cada camada, que serão usados no passo seguinte
2. **Backward Pass:** Também conhecido como retropropagação, é o processo de calcular os gradientes de cada peso e bias da rede neural com respeito a função de custo, usando o algoritmo de retropropagação

3. **Atualização dos pesos:** Depois de obter os gradientes, basta usar um método de otimização, como o SGD, para atualizar os pesos e bias da rede neural

4.4.1 Forward Pass

O forward pass é o processo de passar os dados pela rede neural e armazenar os resultados de cada camada. Para isso, basta seguir a estrutura matemática que vimos anteriormente

```
1 function forward_pass( $X \in \mathbb{R}^{N \times D}$ ,  $W$ ,  $b$ ) {  
2    $Z^{(0)} \leftarrow X$   
3   for  $j = 1, \dots, K + 1$  do  
4      $Z^{(j)} \leftarrow h^{(j)}(Z^{(j-1)}W^{(j)} + b^{(j)}\mathbb{1}^T)$   
5   end for  
6   return  $Z^{(K+1)}$   
7 }
```

4.4.2 Backpropagation

O backward pass é o processo de calcular os gradientes de cada peso e bias da rede neural com respeito a função de custo, usando o algoritmo de retropropagação. Para isso, basta usar a regra da cadeia para calcular os gradientes de cada camada, começando pela última camada e indo para a primeira camada. Para calcular o gradiente das camadas, antes de tudo precisamos de uma função de perda, e isso vai variar dependendo do problema que estamos tentando resolver. Por exemplo, para um problema de regressão, podemos usar o MSE como função de perda, enquanto para um problema de classificação, podemos usar a entropia cruzada. Depois de escolher a função de perda, basta seguir o algoritmo de retropropagação para calcular os gradientes de cada camada.

Lembrando que, nesse momento, nós temos armazenado/sabemos os valores de:

- $Z^{(j)} \forall j \in \{1, \dots, K + 1\}$
- $W^{(j)} \forall j \in \{1, \dots, K + 1\}$
- $b^{(j)} \forall j \in \{1, \dots, K + 1\}$

4.4.2.1 Regressão

Em redes neurais, a regressão linear é obtida quando a função de ativação da última camada é a função identidade, ou seja, $h^{(K+1)}$ é a função identidade. Nesse caso, a saída da rede neural é dada por:

$$Z^{(K+1)} = Z^{(K)}W^{(K+1)} + b^{(K+1)}\mathbb{1}^T \quad (61)$$

ou seja, **não** é uma regressão linear na última camada como conhecemos, mas uma combinação linear dos valores da saída da camada anterior. Intuitivamente, é como se a rede neural aprendesse uma forma mais fácil dos dados, ao ponto que apenas uma combinação linear dos valores da última camada fosse suficiente para fazer uma boa predição

Para demonstrar o algoritmo, vamos usar o MSE como função de perda. Então, antes de tudo, vamos ter $X \in \mathbb{R}^{N \times D}$ que são meus dados e $T \in \mathbb{R}^{N \times L}$ que é a matriz de resultados que quero prever. Ao aplicar o forward pass, obtemos $Y = Z^{(K+1)} \in \mathbb{R}^{N \times L}$ que é a matriz de predições da minha rede neural. Com isso, podemos calcular a função de perda como:

$$E_n(w) = \frac{1}{2} \sum_k (y_{nk} - t_{nk})^2 \quad (62)$$

Que é o erro quadrático médio para o n -ésimo exemplo de treinamento e o erro total é dado por:

$$E(w) = \sum_{n=1}^N E_n(w) = \|Y - T\|_F^2 \quad (63)$$

O gradiente do erro total com respeito a Y é dado por:

$$\frac{\partial E_n}{\partial y_{nk}} = y_{nk} - t_{nk} \Rightarrow \nabla_Y E(w) = Y - T \quad (64)$$

4.4.2.2 Camadas Escondidas

Mostramos antes como calcular o gradiente da última camada na situação de uma **regressão linear**, mas saiba que pode variar dependendo das função de saída que você escolher. Agora vamos olhar como calcular a derivada das camadas escondidas.

Pegando o erro total

$$E(w) = \sum_{n=1}^N E_n(w) \quad (65)$$

vamos tirar a derivada do erro de acordo com a entrada $w_{ij}^{(p)}$, vamos primeiro considerar o erro linha-a-linha, ou seja, E_n e depois somar os resultados para obter o gradiente total:

$$\frac{\partial E_n}{\partial w_{ij}^{(p)}} = \frac{\partial E_n}{\partial z_{nj}^{(p)}} \frac{\partial z_{nj}^{(p)}}{\partial a_{nj}^{(p)}} \frac{\partial a_{nj}^{(p)}}{\partial w_{ij}^{(p)}} \quad (66)$$

E eu posso fazer essa regra da cadeia pois nada anterior a $w_{ij}^{(p)}$ depende dele, porém, tudo que vem depois, tem uma influência que ele aplicou em sua camada. Agora vamos calcular cada um desses termos:

$$a_{nj}^{(p)} = \sum_{k=1}^{M_{p-1}} z_{nk}^{(p-1)} w_{kj}^{(p)} + b_j^{(p)} \quad (67)$$

$$\frac{\partial a_{nj}^{(p)}}{\partial w_{ij}^{(p)}} = z_{ni}^{(p-1)}$$

$$z_{nj}^{(p)} = h^{(p)}(a_{nj}^{(p)})$$

$$\frac{\partial z_{nj}^{(p)}}{\partial a_{nj}^{(p)}} = h'^{(p)}(a_{nj}^{(p)}) \quad (68)$$

Agora a parte mais delicada, qual é a derivada de $(\partial E_n) / (\partial z_{nj}^{(p)})$? Por enquanto, vamos apenas atribuir um novo nome para ele:

$$\frac{\partial E_n}{\partial z_{nj}^{(p)}} = \delta_{nj}^{(p)} \quad (69)$$

perfeito! Jaja voltamos nele, agora vamos calcular o gradiente TOTAL em todas as linhas:

$$\frac{\partial E}{\partial w_{ij}^{(p)}} = \sum_{n=1}^N \frac{\partial E_n}{\partial w_{ij}^{(p)}} = \sum_{n=1}^N \delta_{nj}^{(p)} h'^{(p)}(a_{nj}^{(p)}) z_{ni}^{(p-1)} \quad (70)$$

legal, obtemos uma fórmula para a derivada do erro total com respeito a $w_{ij}^{(p)}$, mas ainda não sabemos o que é $\delta_{nj}^{(p)}$. Para isso, vamos usar a regra da cadeia para expressar $\delta_{nj}^{(p)}$ em termos de $\delta_{nk}^{(p+1)}$, como assim? Lembra que, quando eu calculo $z_{nj}^{(p)}$, eu vou mandar ele pra próxima camada, e a próxima camada vai usar ele para calcular $a_{nk}^{(p+1)}$ e depois $z_{nk}^{(p+1)}$. Então eu teria algo assim:

$$\begin{aligned} z_{nk}^{(p+1)} &= h^{(p+1)}\left(a_{nk}^{(p+1)}\right) \\ a_{nk}^{(p+1)} &= \sum_{j=1}^{M_p} z_{nj}^{(p)} w_{jk}^{(p+1)} + b_k^{(p+1)} \end{aligned} \quad (71)$$

olha só, o meu $z_{nj}^{(p)}$ ta ali no meio do somatório, e porque diabos isso me é útil? Calma que ainda vai fazer sentido. Com essa estrutura, concorda comigo que meu $z_{nj}^{(p)}$ não ta só no neurônio k , mas também ta no neurônio 1, 2, ..., M_{p+1} (ou seja, eu mando ele pra todos os neurônios da camada seguinte)? Então, quando eu calcular a derivada de E_n com respeito a $z_{nj}^{(p)}$, eu vou ter que somar a contribuição de cada um desses neurônios, ou seja:

$$\frac{\partial E_n}{\partial z_{nj}^{(p)}} = \sum_{k=1}^{M_{p+1}} \frac{\partial E_n}{\partial z_{nk}^{(p+1)}} \frac{\partial z_{nk}^{(p+1)}}{\partial a_{nk}^{(p+1)}} \frac{\partial a_{nk}^{(p+1)}}{\partial z_{nj}^{(p)}} \quad (72)$$

mas perceba que:

$$\begin{aligned} a_{nk}^{(p+1)} &= \sum_{j=1}^{M_p} z_{nj}^{(p)} w_{jk}^{(p+1)} + b_k^{(p+1)} \\ \Rightarrow \frac{\partial a_{nk}^{(p+1)}}{\partial z_{nj}^{(p)}} &= w_{jk}^{(p+1)} \end{aligned} \quad (73)$$

$$\frac{\partial z_{nk}^{(p+1)}}{\partial a_{nk}^{(p+1)}} = h'^{(p+1)}\left(a_{nk}^{(p+1)}\right) \quad (74)$$

e pela definição de δ que fizemos antes, temos que:

$$\frac{\partial E_n}{\partial z_{nk}^{(p+1)}} = \delta_{nk}^{(p+1)} \quad (75)$$

logo:

$$\frac{\partial E_n}{\partial z_{nj}^{(p)}} = \sum_{k=1}^{M_{p+1}} \delta_{nk}^{(p+1)} h'^{(p+1)}\left(a_{nk}^{(p+1)}\right) w_{jk}^{(p+1)} \quad (76)$$

beleza, e porque que isso é útil? Agora eu to escrevendo a derivada de $z_{nj}^{(p)}$ na camada p em função da camada seguinte $p + 1$. Por que isso ajudaria? Simples! Porque nós começamos já calculando o gradiente da última camada, lembra? Ou seja, nós calculamos $\delta_{nk}^{(K+1)}$ para a última camada, e agora, usando a fórmula acima, podemos calcular $\delta_{nj}^{(K)}$ para a camada anterior, e depois $\delta_{ni}^{(K-1)}$ para a camada anterior a essa, e assim por diante, até chegar na primeira camada. Ou seja, nós conseguimos calcular o gradiente de todas as camadas usando apenas o gradiente da última camada e a estrutura da rede neural. E isso é o que chamamos de retropropagação! Mas antes de concluir, vamos reformular rapidinho de uma forma matricial, afinal, não vamos no computador fazer o cálculo de cada neurônio individualmente, mas sim de toda a camada de uma vez.

Para entendermos bem como e porque essa mudança pra matriz funciona, é interessante fazermos um passo-a-passo. Primeiro, na abordagem ingênua, nós atualizamos cada peso individualmente, ou seja:

$$w_{ij}^{(p)} \leftarrow w_{ij}^{(p)} - \eta \frac{\partial E}{\partial w_{ij}^{(p)}} \quad (77)$$

ou seja, olhando numa abordagem mais matricial, mas ainda ingênua, podemos escrever:

$$W^{(p)} \leftarrow W^{(p)} - \eta \begin{pmatrix} \frac{\partial E}{\partial w_{11}^{(p)}} & \frac{\partial E}{\partial w_{12}^{(p)}} & \cdots & \frac{\partial E}{\partial w_{1M_p}^{(p)}} \\ \frac{\partial E}{\partial w_{21}^{(p)}} & \frac{\partial E}{\partial w_{22}^{(p)}} & \cdots & \frac{\partial E}{\partial w_{2M_p}^{(p)}} \\ \vdots & \vdots & & \vdots \\ \frac{\partial E}{\partial w_{M_{p-1}1}^{(p)}} & \frac{\partial E}{\partial w_{M_{p-1}2}^{(p)}} & \cdots & \frac{\partial E}{\partial w_{M_{p-1}M_p}^{(p)}} \end{pmatrix} \quad (78)$$

Essa matriz maior, nós a chamamos de **jacobiana** de E com respeito a $W^{(p)}$, ou seja, $J_{E,W^{(p)}}$. Agora, usando a fórmula que obtivemos para a derivada de cada peso, podemos escrever

$$J_{E,W^{(p)}} = \begin{pmatrix} \sum_{n=1}^N \delta'_{n1} z_{n1}^{(p-1)} & \sum_{n=1}^N \delta'_{n2} z_{n1}^{(p-1)} & \cdots & \sum_{n=1}^N \delta'_{nM_p} z_{n1}^{(p-1)} \\ \sum_{n=1}^N \delta'_{n1} z_{n2}^{(p-1)} & \sum_{n=1}^N \delta'_{n2} z_{n2}^{(p-1)} & \cdots & \sum_{n=1}^N \delta'_{nM_p} z_{n2}^{(p-1)} \\ \vdots & \vdots & & \vdots \\ \sum_{n=1}^N \delta'_{n1} z_{nM_{p-1}}^{(p-1)} & \sum_{n=1}^N \delta'_{n2} z_{nM_{p-1}}^{(p-1)} & \cdots & \sum_{n=1}^N \delta'_{nM_p} z_{nM_{p-1}}^{(p-1)} \end{pmatrix} \quad (79)$$

aqui, por questão de leitura, mas não muda em nada as fórmulas, eu defini $\delta'_{nj} = \delta_{nj}^{(p)} h'^{(p)}(a_{nj}^{(p)})$. Tirando o somatório para fora, temos:

$$J_{E,W^{(p)}} = \sum_{n=1}^N \begin{pmatrix} \delta'_{n1} z_{n1}^{(p-1)} & \delta'_{n2} z_{n1}^{(p-1)} & \cdots & \delta'_{nM_p} z_{n1}^{(p-1)} \\ \delta'_{n1} z_{n2}^{(p-1)} & \delta'_{n2} z_{n2}^{(p-1)} & \cdots & \delta'_{nM_p} z_{n2}^{(p-1)} \\ \vdots & \vdots & & \vdots \\ \delta'_{n1} z_{nM_{p-1}}^{(p-1)} & \delta'_{n2} z_{nM_{p-1}}^{(p-1)} & \cdots & \delta'_{nM_p} z_{nM_{p-1}}^{(p-1)} \end{pmatrix} \quad (80)$$

reparem na matriz acima e nessa definição de produto externo:

$$u = (u_1 \ u_2 \ \dots \ u_m)^T \text{ e } v = (v_1 \ v_2 \ \dots \ v_n)^T$$

$$\Rightarrow uv^T = \begin{pmatrix} u_1 v_1 & u_1 v_2 & \cdots & u_1 v_n \\ u_2 v_1 & u_2 v_2 & \cdots & u_2 v_n \\ \vdots & \vdots & & \vdots \\ u_m v_1 & u_m v_2 & \cdots & u_m v_n \end{pmatrix} \quad (81)$$

então definindo $\delta'_n = (\delta'_{n1} \ \delta'_{n2} \ \dots \ \delta'_{nM_p})^T$ e $z_n^{(p-1)} = (z_{n1}^{(p-1)} \ z_{n2}^{(p-1)} \ \dots \ z_{nM_{p-1}}^{(p-1)})^T$, temos que:

$$J_{E,W^{(p)}} = \sum_{n=1}^N \delta'_n (z_n^{(p-1)})^T \quad (82)$$

agora, reparando melhor em δ'_n , vamos abrir sua definição:

$$\begin{aligned}\delta_n^{(p)} &= \left(\delta_{n1}^{(p)} \quad \delta_{n2}^{(p)} \quad \dots \quad \delta_{nM_p}^{(p)} \right)^T \\ &= \left(\delta_{n1}^{(p)} h'^{(p)}(a_{n1}^{(p)}) \quad \delta_{n2}^{(p)} h'^{(p)}(a_{n2}^{(p)}) \quad \dots \quad \delta_{nM_p}^{(p)} h'^{(p)}(a_{nM_p}^{(p)}) \right)^T\end{aligned}\quad (83)$$

então definindo $h'^{(p)}(a_n^{(p)}) = (h_{n1}^{(p)} \quad h_{n2}^{(p)} \quad \dots \quad h_{nM_p}^{(p)})^T$ onde $h_{nj}^{(p)} = h'^{(p)}(a_{nj}^{(p)})$, também podemos escrever assim:

$$\delta_n^{(p)} = \delta_n^{(p)} \odot h'^{(p)}(a_n^{(p)}) \quad (84)$$

onde $\odot$ é o produto de Hadamard, ou seja, o produto elemento a elemento. Então, finalmente, temos que:

$$J_{E, W^{(p)}} = \sum_{n=1}^N \left(\delta_n^{(p)} \odot h'^{(p)}(a_n^{(p)}) \right) (z_n^{(p-1)})^T \quad (85)$$

ok, mas tem como simplificar MAIS AINDA. Por questão de leitura, vamos escrever usando δ' em vez de $\delta \cdot h'$. Se vocês lembram, em álgebra linear, existe um teorema chamado **decomposição em soma de matrizes de rank 1**

Teorema 4.4.2.2.1 (Decomposição em soma de matrizes de posto 1): Dada uma matriz $A \in \mathbb{R}^{m \times n}$ com colunas $a_1, a_2, \dots, a_n$ e $B \in \mathbb{R}^{n \times p}$ com linhas $b_1^T, b_2^T, \dots, b_p^T$, então o produto AB é dado por:

$$\left(\begin{array}{c|c|c|c} | & | & \dots & | \\ a_1 & a_2 & \dots & a_n \\ | & | & \dots & | \end{array} \right) \begin{pmatrix} - & b_1^T & - \\ - & b_2^T & - \\ & \vdots & \\ - & b_p^T & - \end{pmatrix} = \sum_{i=1}^n a_i b_i^T \quad (86)$$

perceba como essa é a EXATA MESMA ESTRUTURA de $J_{E, W^{(p)}}$. Então, se definirmos $\Delta'^{(p)}$ sendo a matriz com **linhas** $\delta_n^{(p)}$ e $Z^{(p-1)}$ sendo a matriz com **linhas** $z_n^{(p-1)}$, temos que:

$$J_{E, W^{(p)}} = (Z^{(p-1)})^T \Delta'^{(p)} \quad (87)$$

não só isso, mas também, para calcular $\Delta'^{(p)}$, vamos recapitular a fórmula de um $\delta_n^{(p)}$:

$$\delta_n^{(p)} = \sum_{k=1}^{M_{p+1}} \delta_{nk}^{(p+1)} w_{jk}^{(p+1)} \quad (88)$$

então, definindo $W^{(p+1)}$ como a matriz de pesos da camada seguinte onde cada **coluna** é o vetor de pesos de um neurônio da camada seguinte, temos que:

$$\begin{aligned}\delta_n^{(p)} &= \sum_{k=1}^{M_{p+1}} \delta_{nk}^{(p+1)} w_j^{(p+1)} \\ \Rightarrow \Delta'^{(p)} &= \Delta'^{(p+1)} (W^{(p+1)})^T\end{aligned}\quad (89)$$

e para finalizar, precisamos também do resultado do gradiente com respeito ao bias, que é dado por:

$$\begin{aligned}
\frac{\partial E}{\partial b_j^{(p)}} &= \sum_{n=1}^N \frac{\partial E_n}{\partial b_j^{(p)}} = \sum_{n=1}^N \frac{\partial E_n}{\partial z_{nj}^{(p)}} \frac{\partial z_{nj}^{(p)}}{\partial a_{nj}^{(p)}} \frac{\partial a_{nj}^{(p)}}{\partial b_j^{(p)}} \\
a_{nj}^{(p)} &= \sum_{k=1}^{M_{p-1}} z_{nk}^{(p-1)} w_{kj}^{(p)} + b_j^{(p)} \Rightarrow \frac{\partial a_{nj}^{(p)}}{\partial b_j^{(p)}} = 1 \\
&\Rightarrow \frac{\partial E}{\partial b_j^{(p)}} = \sum_{n=1}^N \delta'_{nj}^{(p)}
\end{aligned} \tag{90}$$

ou seja, definindo $\Delta'^{(p)}$ como a matriz com **colunas** $\delta'_{nj}^{(p)}$, temos que:

$$J_{E,b^{(p)}} = \sum_{n=1}^N \delta'_{nj}^{(p)} = \sum_{n=1}^N \left(\delta_n^{(p)} \odot h'^{(p)}(a_n^{(p)}) \right) \tag{91}$$

4.4.3 O Algoritmo Completo

Multilayer Perceptron

```

1  $\mathbf{W} \leftarrow [\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(P+1)}]$ 
2  $\mathbf{b} \leftarrow [b^{(1)}, \dots, b^{(P+1)}]$ 
3  $\mathbf{h} \leftarrow [h^{(1)}, \dots, h^{(P+1)}]$ 
4  $\mathbf{h}' \leftarrow [h'^{(1)}, \dots, h'^{(P+1)}]$ 
5  $\mathbf{A} \leftarrow [A^{(1)} = 0, \dots, A^{(P+1)} = 0]$ 
6  $\mathbf{Z} \leftarrow [Z^{(1)} = 0, \dots, Z^{(P+1)} = 0]$ 
7 // Forward pass
8 for  $j = 1, \dots, P + 1$  do
9    $A^{(j)} \leftarrow Z^{(j-1)} \mathbf{W}^{(j)} + b^{(j)} \mathbf{1}^T$ 
10   $Z^{(j)} \leftarrow h^{(j)}(A^{(j)})$ 
11 end for
12 // Backward pass
13  $\Delta'^{(P+1)} \leftarrow \nabla_Z E \odot h'^{(P+1)}(A^{(P+1)})$ 
14 for  $j = P, P - 1, \dots, 1$  do
15    $\Delta'^{(j)} \leftarrow \Delta'^{(j+1)} (\mathbf{W}^{(j+1)})^T \odot h'^{(j)}(A^{(j)})$ 
16 end for

```

Multilayer Perceptron 2: Multilayer Perceptron

4.4.4 Exemplo em código

Para um exemplo em código, acesse [Machine Learning - João Pedro Jerônimo](#)

Inferência Variacional

5.1 Introdução

Na inferência variacional, nosso objetivo é aproximar uma distribuição p através de outra distribuição q que conseguimos manipular mais facilmente. Fazemos isso minimizando alguma medida de discrepância entre as duas distribuições, a forma mais comum de fazer isso é através da divergência de Kullback-Leibler, que é dada por:

$$\text{KL}(q\|p) = \int q(x) \ln \left\{ \frac{q(x)}{p(x)} \right\} dx = \mathbb{E}_{x \sim q} \left[\ln \left\{ \frac{q(x)}{p(x)} \right\} \right] \quad (92)$$

5.2 Propriedades da divergência de Kullback-Leibler

Teorema 5.2.1 (Não-negatividade): A divergência de Kullback-Leibler é não-negativa, ou seja, $\text{KL}(q\|p) \geq 0$

Demonstração: A desigualdade de Jensen fala que, para uma função convexa f e uma variável aleatória X , temos que:

$$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)] \quad (93)$$

sabemos que $-\ln(x)$ é uma função convexa, então, aplicando a desigualdade de Jensen, temos que:

$$-\ln \left(\mathbb{E}_{x \sim q} \left[\frac{p(x)}{q(x)} \right] \right) \leq \mathbb{E}_{x \sim q} \left[-\ln \left(\frac{p(x)}{q(x)} \right) \right] = \mathbb{E}_{x \sim q} \left[\ln \left(\frac{q(x)}{p(x)} \right) \right] = \text{KL}(q\|p) \quad (94)$$

porém:

$$\mathbb{E}_{x \sim q} \left[\frac{p(x)}{q(x)} \right] = \int q(x) \left\{ \frac{p(x)}{q(x)} \right\} dx = \int p(x) dx = 1 \quad (95)$$

logo:

$$-\ln \left(\mathbb{E}_{x \sim q} \left[\frac{p(x)}{q(x)} \right] \right) = -\ln(1) = 0 \quad (96)$$

chegando à conclusão que:

$$0 \leq \text{KL}(q\|p) \quad (97)$$

□

Teorema 5.2.2 (Identidade):

$$\text{KL}(q\|p) = 0 \Leftrightarrow q = p \text{ quase certamente} \quad (98)$$

ou seja, a divergência de Kullback-Leibler é zero se, e somente se, q e p são iguais

Demonstração: $\Rightarrow$) Se $q = p$, então:

$$\text{KL}(q\|p) = \int q(x) \ln \left\{ \frac{q(x)}{p(x)} \right\} dx = \int q(x) \ln \{1\} dx = \int q(x) 0 dx = 0 \quad (99)$$

$\Leftarrow$) Se $KL(q\|p) = 0$, então:

$$\mathbb{E}_{x \sim q}[\ln q(x)] = \mathbb{E}_{x \sim q}[\ln p(x)] \quad (100)$$

como $-\ln$ é uma função estritamente convexa, então, pela desigualdade de Jensen, temos que:

$$0 \leq -\mathbb{E}_{x \sim q} \left[\ln \left(\frac{p(x)}{q(x)} \right) \right] \quad (101)$$

porém, a desigualdade de Jensen enuncia que:

$$\mathbb{E}[f(X)] = f(\mathbb{E}[X]) \Leftrightarrow X = c \quad (102)$$

ou seja, temos que

$$\frac{p(x)}{q(x)} = c \quad (103)$$

entretanto ambas são densidades, assim:

$$\int q(x)c \, dx = c = \int p(x) \, dx = 1 \Rightarrow c = 1 \Rightarrow \frac{p(x)}{q(x)} = 1 \Rightarrow p(x) = q(x) \quad (104)$$

□

Teorema 5.2.3 (KL e Entropia Cruzada): A divergência de Kullback-Leibler pode ser escrita como a diferença entre a entropia cruzada e a entropia de q , ou seja:

$$KL(q\|p) = H(q, p) - H(q) \quad (105)$$

onde $H(q, p) = -\mathbb{E}_{x \sim q}[\ln p(x)]$ é a entropia cruzada entre q e p e $H(q) = -\mathbb{E}_{x \sim q}[\ln q(x)]$ é a entropia de q . Um ótimo vídeo que fala sobre isso é o [The Key Equation Behind Probability](#) do canal Artem Kirsanov

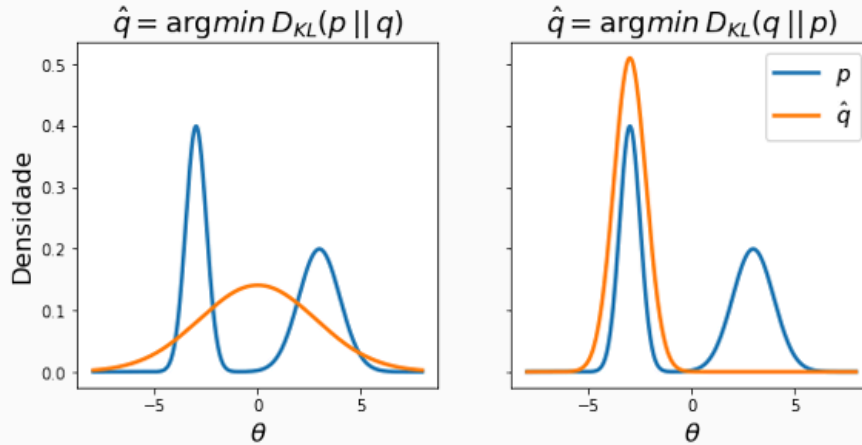
Demonstração:

$$\begin{aligned} KL(q\|p) &= \int q(x) \ln \left\{ \frac{q(x)}{p(x)} \right\} dx \\ &= \int q(x) \ln \{q(x)\} dx - \int q(x) \ln \{p(x)\} dx \\ &= -H(q) + H(q, p) \\ &= H(q, p) - H(q) \end{aligned} \quad (106)$$

□

Divergência KL forward vs reverse: É importante notar que, de forma geral, a divergência KL não é simétrica — i.e., $KL(q\|p) \neq KL(p\|q)$. Na literatura de ML, é comum chamar $KL(q\|p)$ de divergência KL reversa. Conversamente, $KL(p\|q)$ é conhecida como a divergência forward. Empiricamente, é bem estabelecido que minimizar a divergência reversa costuma resultar em resultados que focam em alguma(s) modas. Por outro lado, minimizar a divergência forward

costuma promover aproximações que cobrem melhor o suporte de p . A figura abaixo ilustra esse fenômeno com $p(\theta) = \frac{1}{2}\mathcal{N}(\theta|3, 1) + \frac{1}{2}\mathcal{N}(\theta|-3, (\frac{1}{2})^2)$ e q sendo Gaussiana univariada



5.3 Cota Inferior (ELBO)

Normalmente estamos realizando inferência bayesiana, ou seja, queremos calcular a distribuição posterior $p(\theta|D)$, mas isso é difícil de fazer diretamente. Então, vamos usar a divergência KL para encontrar uma aproximação $q(\theta)$ para $p(\theta|D)$. Para isso, vamos minimizar a divergência KL reversa:

$$\begin{aligned}
 \text{KL}(q(\theta)||p(\theta|D)) &= \mathbb{E}_{\theta \sim q(\theta)} \left[\ln \left\{ \frac{q(\theta)}{p(\theta|D)} \right\} \right] \\
 &= \mathbb{E}_{\theta \sim q(\theta)} [\ln q(\theta)] - \mathbb{E}_{\theta \sim q(\theta)} [\ln p(\theta|D)] \\
 &= \mathbb{E}_{\theta \sim q(\theta)} [\ln q(\theta)] - \mathbb{E}_{\theta \sim q(\theta)} \left[\ln \frac{p(D|\theta)p(\theta)}{p(D)} \right] \\
 &= \mathbb{E}_{\theta \sim q(\theta)} [\ln q(\theta)] - \mathbb{E}_{\theta \sim q(\theta)} [\ln p(D, \theta)] + \ln p(D)
 \end{aligned} \tag{107}$$

Vamos definir $L(q)$ como:

$$L(q) = \mathbb{E}_{\theta \sim q(\theta)} [\ln p(D, \theta)] - \mathbb{E}_{\theta \sim q(\theta)} [\ln q(\theta)] \tag{108}$$

então temos que a distribuição q que minimiza a divergência KL seria:

$$\begin{aligned}
 \hat{q} &= \text{argmin}_q \text{KL}(q(\theta)||p(\theta|D)) \\
 &= \text{argmin}_q \mathbb{E}_{\theta \sim q(\theta)} [\ln q(\theta)] - \mathbb{E}_{\theta \sim q(\theta)} [\ln p(D, \theta)] + \ln p(D) \\
 &= \text{argmin}_q -L(q) + \ln p(D) \\
 &= \text{argmax}_q L(q)
 \end{aligned} \tag{109}$$

algo interessante de se ressaltar é que, como $\text{KL} \geq 0$, temos que:

$$\ln p(D) - L(q) = \text{KL} \geq 0 \Rightarrow L(q) \leq \ln p(D) \tag{110}$$

logo, $\ln p(D)$ (conhecido como evidência) é uma cota superior para $L(q)$, e como queremos maximizar $L(q)$, estamos na verdade tentando encontrar a melhor aproximação para a evidência. Por isso, chamamos $L(q)$ de **Evidence Lower Bound** (ELBO), ou cota inferior da evidência.

5.4 Lidando com ELBO intratável

Com relação à escolha de Q , costuma-se adotar uma família de distribuições paramétricas. Deste modo, podemos maximizar L sobre um espaço de parâmetros Ω . Por exemplo, se Q é o conjunto das distribuições Gaussianas univariadas, $\Omega = \mathbb{R} \times \mathbb{R}^+$ e $\omega \in \Omega$ é um par média/variância. De maneira geral, os termos envolvidos no ELBO podem ser intratáveis e, até meados da década passada, desenvolver soluções customizadas para posteriors diferentes era considerado uma contribuição técnica em ML.

Atualmente, existem metodologias genéricas, que viabilizam inferência variacional quase como uma tecnologia *off-the-shelf*. A mais famosa dentre essas, é o truque reparametrização. Essa técnica assume que é possível descrever a amostragem de $\theta \sim q$ a partir de uma transformação g de uma variável aleatória auxiliar ε . Além disso, precisamos que g seja diferenciável com respeito aos parâmetros ω de q . Por exemplo, se q é uma distribuição normal com parâmetros $\omega = (\mu, \sigma^2)$, podemos obter uma amostra $\theta \sim q$ definindo $\theta = g(\varepsilon; \omega) = \sigma\varepsilon + \mu$ e amostrando ε de numa gaussiana padrão — i.e., com média zero e variância um.

Com isso, podemos aproximar os termos do ELBO amostrando M variáveis auxiliares $\varepsilon(1), \dots, \varepsilon(M)$ e estimar o gradiente de $L(q)$ com respeito a ω como:

$$\nabla_{\omega} L(q) = \nabla_{\omega} \frac{1}{M} \sum_{m=1}^M \ln p(D|\theta^{(m)}) + \ln p(\theta^{(m)}) - \ln q(\theta^{(m)}; \omega) \quad (111)$$

Note que, na notação acima, q também depende diretamente de ω . Em posse dessa estimativa, podemos utilizar nosso algoritmo de gradiente preferido para otimizar o ELBO. Naturalmente, essa aproximação deve ser refeita com novas amostras a cada passo de gradiente. Para evitar incluir restrições explícitas para garantir que parâmetros restritos sejam válidos (e.g., variâncias devem ser não-negativas), nós também as modelamos como a transformação de uma variável real. No caso citado acima, podemos ter $\omega = (\mu, \sigma^2 = u(z))$ com $u(z) = e^z$ ou $u(z) = \beta^{-1} \ln(1 + e^{\beta z})$ para algum $\beta > 0$.